

Evidencia de Practica

Dialog ? X

Expresión infija:

23+

5

Dialog ? X

Expresión infija:

234*+

14

Dialog

?

×

Expresión infija:

(2+3)*4

Convertir

Evaluar

23+4*

20

Dialog

?

×

Expresión infija:

2*(3+4)

Convertir

Evaluar

234+*

14



Evidencia de Prompt

Actúa como asistente de programación para una práctica de Estructura de Datos en Python.

Contexto: Ya tenemos el proyecto desarrollado en clase con Python, programación orientada a objetos y la interfaz gráfica creada con Qt Designer. Como ya sabes, el proyecto ya permite convertir una expresión infija a posfija utilizando una clase Stack.

Objetivo: Necesito complementar el proyecto para evaluar una expresión posfija utilizando la misma clase Stack. En esta práctica se usarán expresiones numéricas con operandos de un solo dígito, por ejemplo $2 + 3 * 4$, cuya expresión posfija sería $234*+$.

Instrucciones importantes:

1. No generes un proyecto nuevo desde cero.
2. No reemplaces la clase Stack existente.
3. No uses eval() de Python.
4. No generes una nueva clase para el proceso de evaluación, utiliza la clase ya existente y sólo crea un nuevo método
5. Respeta la organización actual del proyecto.
6. La solución debe estar hecha con programación orientada a objetos.
7. La interfaz gráfica debe seguir usando el diseño creado con Qt Designer.
8. Se debe generar un nuevo botón para la evaluación y su etiqueta del resultado y tiene que ser capaz de arrojar el resultado de la anotación infija del usuario, aunque no se utilice el botón de convertir
9. Propón cambios mínimos para agregar la evaluación posfija.
10. Explica en qué archivo y en qué parte debo integrar el código.
11. Explica cómo se usa la pila durante la evaluación.
12. La evaluación debe recorrer la expresión posfija, apilar los dígitos como números y aplicar las operaciones cuando encuentre operadores.
13. Proporciona pruebas para validar el funcionamiento. Como ya sabrás, el código existente es: `from estructuras.lineales.nodo import Node`.

```

from estructuras.lineales.nodo import Node

class Stack:

    def __init__(self):
        self.top = None

    def push(self, data):
        new_node = Node(data)

        if self.top is None:
            self.top = new_node
        else:
            new_node.next = self.top
            self.top = new_node

    def pop(self):
        if self.top == None:
            return "No hay elementos en la pila"
        else:
            temp = self.top.data
            self.top = self.top.next
            return f"El dato que se eliminó de la pila fue: {temp}"

    def top_of_stack(self):
        if self.top == None:
            return "No hay elementos en la pila"
        else:
            return f"El top de la pila es: {self.top.data}"

    def print_stack(self):
        self.x = "No hay elementos en la pila"

        if self.top == None:
            return self.x
        else:
            temp = self.top
            self.x = "top "

```

```

        while temp is not None:
            self.x += f"-> {temp.data} "
            temp = temp.next
        return self.x

    def is_empty(self):
        return self.top is None

    def peek(self):
        if self.top is None:
            return None
        return self.top.data

    def pop_data(self):
        if self.top is None:
            return None
        temp = self.top.data
        self.top = self.top.next
        return temp

```

```

from estructuras.lineales.Pila_Stack import Stack

```

```

class ConversorPosfija:

```

```

    def __init__(self):
        self.pila = Stack()

    def prioridad(self, operador):
        prioridades = {
            '$': 3,
            '*': 2,
            '/': 2,
            '+': 1,
            '-': 1
        }

```

```

        return prioridades.get(operador, 0)
def convertir(self, expresion):
    salida = ""
    for caracter in expresion:
        if caracter.isalnum():
            salida += caracter
        elif caracter == "(":
            self.pila.push(caracter)
        elif caracter == ")":
            while (not self.pila.is_empty()) and self.pila.peek() != "(":
                salida += self.pila.pop_data()
            self.pila.pop_data()
        else:
            while (not self.pila.is_empty()) and self.pila.peek() != "(" and \
                self.prioridad(self.pila.peek()) >= self.prioridad(caracter):
                salida += self.pila.pop_data()
            self.pila.push(caracter)
    while not self.pila.is_empty():
        salida += self.pila.pop_data()
    return salida

import sys
from PyQt5.QtWidgets import QApplication
from load.load_menu import LoadMenu
def main():
    app = QApplication(sys.argv)
    menu = LoadMenu()
    menu.show()
    sys.exit(app.exec_())
if __name__ == "__main__":
    main()

```

```
from PyQt5.QtWidgets import QDialog
from PyQt5 import uic
from estructuras.lineales.aplicacion import ConversorPosfija
```

```
class DialogAplicacion(QDialog):
    def __init__(self):
        super().__init__()
        uic.loadUi("ui/aplicacion.ui", self)
        self.conversor = ConversorPosfija()
        self.btn_conver.clicked.connect(self.convertir)
    def convertir(self):
        expresion = self.txt_infja.text()
        self.resultado = self.conversor.convertir(expresion)
        self.lbl_posfija.setText(self.resultado)
```

Resultado esperado:

- Primero analiza la estructura del código existente.
- Después propón la clase o método para evaluar la expresión posfija numérica.
- Luego indica cómo conectar ese método con el botón de la interfaz gráfica.
- Finalmente proporciona pruebas para comprobar que funciona.
- No omitas la explicación, porque debo entender el código para defenderlo en clase.

Evidencia de Respuesta

1. Análisis de la estructura del código existente

La práctica anterior quedó bien estructurada.

Actualmente el proyecto tiene:

- Una clase Stack que implementa la pila mediante listas enlazadas.
- Una clase ConversorPosfija que contiene un objeto Stack.
- Un método convertir() que transforma una expresión infija en una expresión posfija.
- Una ventana (DialogAplicacion) encargada de interactuar con la interfaz creada con Qt Designer.

La nueva práctica pide complementar el proyecto, no reemplazarlo.

Además, una instrucción importante dice:

“No generar una nueva clase para el proceso de evaluación, utiliza la clase ya existente y sólo crea un nuevo método.”

Por lo tanto, la evaluación debe agregarse dentro de ConversorPosfija, reutilizando la misma pila.

2. Agregar un nuevo método para evaluar la expresión posfija

Archivo:

estructuras/lineales/aplicacion.py

Dentro de la clase

```
class ConversorPosfija:
```

agrega el siguiente método después de convertir()

```
def evaluar(self, expresion):  
    self.pila = Stack()  
    for caracter in expresion:  
        if caracter.isdigit():  
            self.pila.push(int(caracter))  
        else:  
            operando2 = self.pila.pop_data()  
            operando1 = self.pila.pop_data()  
            if caracter == "+":  
                self.pila.push(operando1 + operando2)  
            elif caracter == "-":  
                self.pila.push(operando1 - operando2)  
            elif caracter == "*":  
                self.pila.push(operando1 * operando2)  
            elif caracter == "/":  
                self.pila.push(operando1 / operando2)  
            elif caracter == "$":  
                self.pila.push(operando1 ** operando2)  
    return self.pila.pop_data()
```

¿Por qué funciona?

En una expresión posfija los operadores aparecen **después** de los operandos.

Ejemplo:

234*+

equivale a

$$2 + (3 * 4)$$

El algoritmo recorre la expresión de izquierda a derecha.

Si encuentra un número:

2

lo mete a la pila.

Pila

2

Después aparece

3

Pila

3

2

Después

4

4

3

2

Al encontrar

*

se sacan dos elementos.

4

3

Se calcula

$$3 * 4 = 12$$

y vuelve a meterse.

12

2

Después aparece

+

Se sacan

12

2

Se calcula

$2+12=14$

Se mete nuevamente.

14

Al terminar la expresión, el único elemento de la pila es el resultado.

3. Cambio mínimo en la interfaz gráfica

La práctica solicita:

Se debe generar un nuevo botón para la evaluación y una etiqueta del resultado.

No es necesario modificar la lógica existente.

Solamente agrega en Qt Designer:

Botón

btn_evaluar

Texto:

Evaluar

Etiqueta

lbl_resultado

Texto inicial:

Resultado:

4. Conectar el botón

En

load/load_aplicacion.py

(actualmente tu archivo mostrado)

Dentro del constructor agrega:

```
self.btn_evaluar.clicked.connect(self.evaluar)
```

Quedaría así:

```
self.conversor = ConversorPosfija()
```

```
self.btn_conver.clicked.connect(self.convertir)
```

```
self.btn_evaluar.clicked.connect(self.evaluar)
```

5. Agregar el nuevo método del evento

Debajo de convertir() agrega:

```
def evaluar(self):
```

```
    expresion_infija = self.txt_infja.text()
```

```
    expresion_posfija = self.conversor.convertir(expresion_infija)
```

```
    resultado = self.conversor.evaluar(expresion_posfija)
```

```
    self.lbl_resultado.setText(str(resultado))
```

6. Explicación para defender la práctica

Si el profesor pregunta **¿por qué se utiliza una pila para evaluar una expresión posfija?**, puedes responder lo siguiente:

En una expresión posfija, los operandos aparecen antes que los operadores. La pila permite almacenar temporalmente esos operandos. Cuando se encuentra un operador, se extraen los dos operandos más recientes, se realiza la operación y el resultado se vuelve a insertar en la pila. Este proceso continúa hasta recorrer toda la expresión. Al finalizar, la pila contiene un único elemento, que corresponde al resultado final de la evaluación. La estructura LIFO garantiza que los operandos se recuperen en el orden correcto para cada operación.

Explicación

El código generado por la Inteligencia Artificial fue tomado en su mayoría como lo ha presentad, pero se vio mayormente modificado en las instrucciones durante el prompt que se le presentó, así como la modificación del diseño de la interfaz y sus características. Por un lado, se dio la instrucción de que no se modificara la clase con la que se contaba y simplemente se aplicara un nuevo método o función donde se llevara a cabo el proceso de evaluación:

```
def evaluar(self, expresion):  
    self.pila = Stack()  
    for caracter in expresion:  
        if caracter.isdigit():  
            self.pila.push(int(caracter))  
        else:  
            operando2 = self.pila.pop_data()  
            operando1 = self.pila.pop_data()  
            if caracter == "+":  
                self.pila.push(operando1 + operando2)  
            elif caracter == "-":  
                self.pila.push(operando1 - operando2)  
            elif caracter == "*":  
                self.pila.push(operando1 * operando2)  
            elif caracter == "/":  
                self.pila.push(operando1 / operando2)  
            elif caracter == "$":  
                self.pila.push(operando1 ** operando2)  
    return self.pila.pop_data()
```

El código fue implementado directamente sin ninguna modificación.

El proceso es simple, se lee la entrada que se genere en la caja de texto de Qt y se separa término por término empezando con la primera condición, si era un número, ese número entraría a la pila en el orden en el que se van leyendo, en caso de no ser un número, se extraía el primer número con una variable y el siguiente número de la pila como el

primero y se utilizaba el signo que se encontró para llevar a cabo la operación entre la primera a la segunda variable.

Su lectura es la misma que la anterior, tanto paréntesis, signos y números son tomados en cuenta.

Por otro lado, la interfaz se solicitó que cumpliera la condición de funciona cualquiera de los botones incluso si alguno de los botones era seleccionado, además se agregaron varias modificaciones en el apartado de los nombres de los widgets y algunos de sus procesos, pero sin recibir cambios significativos

```
def evaluar(self):
```

```
    expresion_infija = self.txt_infja.text()
```

```
    expresion_posfija = self.conversor.convertir(expresion_infija)
```

```
    resultado = self.conversor.evaluar(expresion_posfija)
```

```
    self.lbl_evaluacion.setText(str(resultado))
```